
Docente del curso: Benner Humberto Ojeda Rios

Nombres y Apellidos del estudiante:

Fecha: 10/10/2025

Nota:

Indicaciones específicas:

- Duración: **90** minutos
- La evaluación consta de **4** preguntas.
- En caso de haber soluciones similares de preguntas serán calificados con cero (0).
- Sea ordenado y legible en la resolución de cada ejercicio, caso contrario se invalidará su respuesta.

Resuelva:

1. (4 points) **General: Preguntas teóricas y ejercicios**

Para cada afirmación a continuación, indique si es **verdadera o falsa**. Justifique su respuesta. Respuesta **sin justificación** serán calificadas con **CERO**.

1. (0.5 pts) ¿La búsqueda binaria y la secuencial tienen el mismo orden de crecimiento en el peor caso (Θ)?.

2. (0.5 pts) Al insertar en un árbol AVL con n nodos, se necesitan $\Theta(\lg n)$ rotaciones.

3. (0.5 pts) Para cualesquiera constantes $x, y > 1$, se tiene $n^x = O(y^n)$.

4. (0.5 pts) Los árboles AVL y BST siempre tienen el mismo tiempo de ejecución ajustado de *big-O* para insertar elementos.

5. (0.5 pts) Se espera que una función hash no uniforme produzca un peor rendimiento en una tabla hash que una función hash uniforme.

6. (0.5 pts) En un árbol binario “lleno”, la altura está acotada por $c \lg n$ para alguna constante c , sin importar su forma.

7. (0.5 pts) Un árbol AVL está equilibrado, por lo tanto, una mediana de todos los elementos del árbol siempre está en la raíz o en uno de sus dos hijos.

8. (0.5 pts) Suponga que calcular el *hash* de una cadena de longitud k toma tiempo $\Theta(k)$. Dada una cadena de longitud n , queremos hallar los valores *hash* de todas sus subcadenas de longitud k usando un *hash* por división. Podemos hacerlo en tiempo $O(n)$.

2. (5 points) **Notación asintótica**

1. (2 points) Usando la *definición formal* de notación asintótica, demuestre que

$$\log(n!) \in \Theta(n \log n).$$

(Use logaritmos en base e; cualquier otra base cambia solo por una constante.)

Solución:

2. (3 points)

Sea

$$F(n) = \sum_{k=1}^n \sum_{j=1}^k (3j + 2).$$

- a) (1 points) Obtén una **fórmula cerrada** para $F(n)$ (sin sumatorias).
b) (1 points) Usando **límites**, determina el **orden asintótico** de $F(n)$; es decir, encuentra $\Theta(\cdot)$ justificando con

$$\lim_{n \rightarrow \infty} \frac{F(n)}{g(n)} = c \in (0, \infty).$$

- c) (1 points) Concluye formalmente si $F(n) \in \Theta(g(n))$.

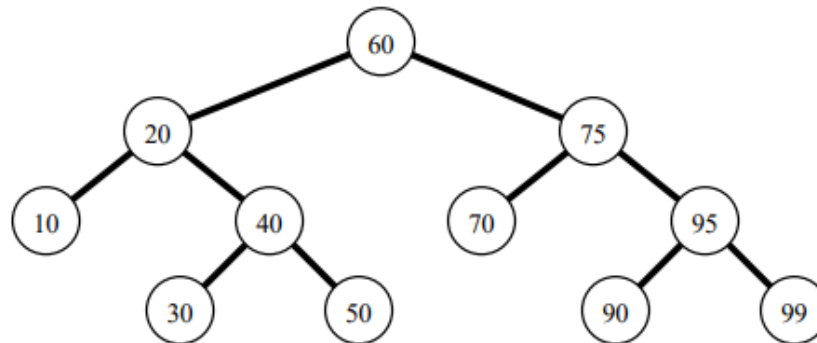
Indicaciones. Puedes usar sin demostración las identidades:

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}, \quad \sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}.$$

Solución:

3. (8 points) **Árboles**

- (a) (1 pt) Supongamos que insertamos 42 en el árbol AVL de abajo usando el algoritmo de inserción de AVL. En la página siguiente, muestra el árbol resultante tras el reequilibrio. También recomendamos mostrar los pasos intermedios para obtener crédito parcial.



Solución:

- (b) (4 pts) Humberto estudia ciencias de la computación en UTEC. Después de haber cursado la disciplina de estructuras de datos a desarrollado una pasión por los arboles balanceados AVL. Humberto le pide que le ayude con la siguiente tarea, dado un arreglo de enteros, su tarea es encontrar una secuencia en la cual esos enteros deberían ser insertados en árbol AVL tal que ninguna rotación es requerida para balancear el árbol. Considere que todos los elementos son distintos.

■ **Ejemplo 1:**

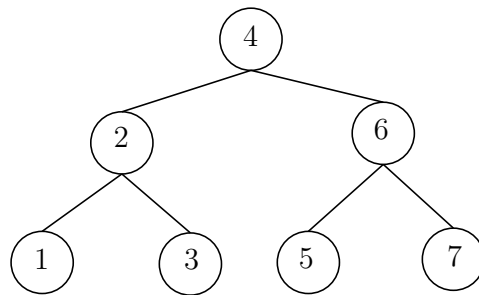
Input:

array = [2, 4, 1, 3, 5, 6, 7]

Output:

[4, 2, 6, 1, 3, 5, 7]

Explicación: La inserción de 4, 2, 6, 1, 3, 5 y 7 genera el árbol de abajo sin la necesidad de utilizar ninguna rotación para balancear el árbol.



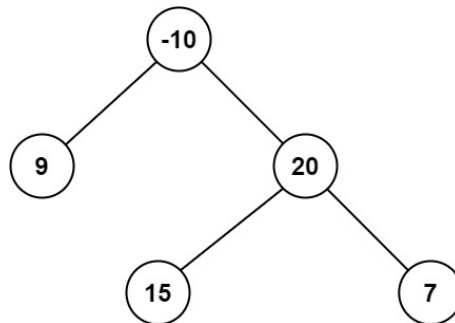
Restricciones:

- El número de elementos en el arreglo estará en el rango $[1, 10^5]$.
- $1 \leq \text{array}[i] \leq 10^5$, para $1 \leq i \leq 10^5$.

Solución:

- (c) (3 pts) Imagina una red eléctrica rural que tiene la forma de un **árbol binario**, donde cada *estación* es un nodo y cada *cable* es una arista que conecta dos estaciones vecinas. Por mantenimiento, **no puedes pasar dos veces por la misma estación** en una misma inspección. Algunas estaciones **aportan energía** (valores positivos) y otras **consumen** (valores negativos). Tu objetivo es planear una ruta que **maximice la energía neta** acumulada. La ruta **no** está obligada a pasar por la estación principal (raíz); puede empezar y terminar en cualquier par de estaciones conectadas por una cadena válida.

Tarea: Dada la **raíz** del árbol binario, **retorna la suma máxima** entre todos los caminos posibles **no vacíos**.



Entrada: root = [-10,9,20,null,null,15,7]

Salida: 42

Explicación: El camino óptimo es 15 -> 20 -> 7 con una suma de camino de $15 + 20 + 7 = 42$.

Restricciones

- El número de nodos en el árbol está en el rango $[1, 3 \times 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$.

Vea el código siguiente para mayor referencia en el diseño de su solución en **ALTO NIVEL**.

```

/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 * };
 */
int maxPathSum(TreeNode* root) { }
```

Solución en ALTO NIVEL: